

# Factory Method Design Pattern

---

## Theory, Application & Refactoring Report

Smart Transportation Management System — Java

**Pattern Category:** Creational Design Pattern

**Language:** Java

**Scope:** Object Creation & Factory Abstraction

Prepared for: Software Design Patterns Course

May 16, 2026

# Contents

---

|          |                                                                              |          |
|----------|------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction to Design Patterns</b>                                       | <b>2</b> |
| <b>2</b> | <b>Factory Method Design Pattern — Theory</b>                                | <b>2</b> |
| 2.1      | What Is the Factory Method Pattern? . . . . .                                | 2        |
| 2.2      | Why Do We Need It? . . . . .                                                 | 2        |
| 2.3      | How Does the Factory Method Pattern Solve This? . . . . .                    | 3        |
| 2.4      | Key Benefits . . . . .                                                       | 3        |
| 2.5      | When Should You Use the Factory Method Pattern? . . . . .                    | 4        |
| 2.6      | Structure of the Factory Method Pattern . . . . .                            | 4        |
| 2.7      | How to Implement the Factory Method Pattern — Step by Step . . . . .         | 4        |
| <b>3</b> | <b>Refactoring the Smart Transportation Management System</b>                | <b>5</b> |
| 3.1      | Overview of Changes . . . . .                                                | 5        |
| 3.2      | Change 1 — Vehicle Interface Kept Intact . . . . .                           | 5        |
| 3.2.1    | Original Code . . . . .                                                      | 5        |
| 3.2.2    | Decision . . . . .                                                           | 5        |
| 3.3      | Change 2 — Concrete Vehicle Classes Improved . . . . .                       | 5        |
| 3.3.1    | Original Code (example: Bus) . . . . .                                       | 5        |
| 3.3.2    | Refactored Code . . . . .                                                    | 6        |
| 3.3.3    | Why This Change? . . . . .                                                   | 6        |
| 3.4      | Change 3 — Abstract Creator: VehicleFactory (New Addition) . . . . .         | 6        |
| 3.4.1    | Code Added . . . . .                                                         | 6        |
| 3.4.2    | Why This Change? . . . . .                                                   | 6        |
| 3.5      | Change 4 — Concrete Creators: Four Factory Classes (New Additions) . . . . . | 7        |
| 3.5.1    | Code Added . . . . .                                                         | 7        |
| 3.5.2    | Why This Change? . . . . .                                                   | 7        |
| 3.6      | Change 5 — Client Code with Menu (New Addition) . . . . .                    | 8        |
| 3.6.1    | Code Added . . . . .                                                         | 8        |
| 3.6.2    | Why This Change? . . . . .                                                   | 8        |
| <b>4</b> | <b>Comparison: Before vs. After Refactoring</b>                              | <b>9</b> |
| <b>5</b> | <b>Design Principles Applied</b>                                             | <b>9</b> |
| <b>6</b> | <b>Conclusion</b>                                                            | <b>9</b> |

# 1 Introduction to Design Patterns

Before understanding the Factory Method Pattern specifically, it is important to understand what design patterns are and why software engineers use them.

## What is a Design Pattern?

A **design pattern** is a reusable, well-tested solution to a commonly occurring problem in software design. It is not finished code you can paste directly into your program — rather, it is a *blueprint* or *template* that describes how to solve a problem in a structured way.

Design patterns were popularised by the “Gang of Four” (GoF) in their 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software*.

Design patterns are grouped into three main categories:

| Category           | Description                                                                         |
|--------------------|-------------------------------------------------------------------------------------|
| <b>Creational</b>  | Deal with <i>object creation</i> . Control how and when objects are instantiated.   |
| <b>Structural</b>  | Deal with <i>object composition</i> . Define how objects and classes are assembled. |
| <b>Behavioural</b> | Deal with <i>communication</i> between objects and responsibilities.                |

The **Factory Method Pattern** belongs to the **Creational** category.

## 2 Factory Method Design Pattern — Theory

### 2.1 What Is the Factory Method Pattern?

#### Definition

The **Factory Method Pattern** defines an interface (or abstract class) for creating an object, but lets *subclasses* decide which class to instantiate. The factory method defers object instantiation to subclasses.

In simple terms: instead of writing `new SomeClass()` directly inside your main or client code, you call a *factory method* that decides which object to create and returns it. The client code never needs to know the details of *how* the object was built.

Think of it like ordering food at a restaurant. You say “I want a burger”, and the kitchen (factory) prepares it for you. You do not need to know the recipe, ingredients, or cooking method. You simply receive the finished product.

### 2.2 Why Do We Need It?

Consider this typical beginner code without any pattern:

```
1 // In main() or client code:
2 if (userChoice.equals("Bus")) {
```

```
3     Vehicle v = new Bus();           // tightly coupled
4 } else if (userChoice.equals("Taxi")) {
5     Vehicle v = new Taxi();         // tightly coupled
6 } else if (userChoice.equals("Scooter")) {
7     Vehicle v = new ElectricScooter(); // tightly coupled
8 }
```

Listing 1: Direct object creation — tightly coupled code

**Problems with the above approach:**

- Every time a new vehicle type is added (e.g., Train, Boat), the `main()` class must be modified.
- Object creation logic is mixed with business logic — hard to read and maintain.
- Unit testing is difficult because `main()` depends on concrete classes.
- Violates the **Open/Closed Principle**: classes should be *open for extension* but *closed for modification*.

## 2.3 How Does the Factory Method Pattern Solve This?

**With the Factory Method Pattern:**

- A dedicated factory class handles object creation.
- The client code only calls `factory.createVehicle()` — it never uses `new` directly.
- Adding a new vehicle means adding a new factory class *only* — existing code is untouched.
- Each factory class has a single responsibility: create one type of object.

## 2.4 Key Benefits

### 1. Loose Coupling

The client code depends on the abstract `VehicleFactory` and `Vehicle` interface, not on any specific class like `Bus` or `Taxi`. Changing the internal implementation of `Bus` has zero impact on the client.

### 2. Open/Closed Principle (OCP)

The system is open for extension (add `TrainFactory`) but closed for modification (you do not touch existing factories or `main()`).

### 3. Single Responsibility Principle (SRP)

Each factory class has exactly one job: creating one type of vehicle. This makes the code easier to understand, test, and maintain.

### 4. Scalability

As the number of vehicle types grows (5, 10, 50), the architecture stays clean. You just keep adding factory classes without restructuring anything.

### 5. Testability

Because factories are separate classes, you can mock or replace them in unit tests without touching production code.

## 6. Readability

The intent of the code becomes clear. `new BusFactory().createVehicle()` reads like plain English: “use the bus factory to create a vehicle.”

## 2.5 When Should You Use the Factory Method Pattern?

Use this pattern when:

- You do not know ahead of time which class you will need to instantiate.
- You want subclasses to control which objects are created.
- Your code creates objects of the same family (vehicles, shapes, users) but the exact type varies.
- You want to centralise and encapsulate creation logic.
- You anticipate that the number of concrete types will grow over time.

## 2.6 Structure of the Factory Method Pattern

The pattern consists of four roles:

| Role                    | Also Called      | Responsibility                                            |
|-------------------------|------------------|-----------------------------------------------------------|
| <b>Product</b>          | Interface        | Defines the common interface all products must follow     |
| <b>Concrete Product</b> | Implementation   | The real object that is created (Bus, Taxi, etc.)         |
| <b>Creator</b>          | Abstract Factory | Declares the factory method; may also define shared logic |
| <b>Concrete Creator</b> | Concrete Factory | Overrides the factory method to return a specific product |

## 2.7 How to Implement the Factory Method Pattern — Step by Step

### Implementation Guide

**Step 1:** Define a **Product Interface** that all products will implement.

**Step 2:** Create **Concrete Product** classes that implement the interface.

**Step 3:** Create an **Abstract Creator** (abstract class) with an abstract `createProduct()` factory method.

**Step 4:** Create **Concrete Creator** subclasses, each overriding `createProduct()` to return one specific product.

**Step 5:** In the **Client Code**, use the abstract creator type and call the factory method. Never use `new ConcreteProduct()` directly.

## 3 Refactoring the Smart Transportation Management System

### 3.1 Overview of Changes

The original code provided a working system with a `Vehicle` interface and four concrete vehicle classes. However, it lacked any structural design and left object creation entirely unmanaged. The following table summarises all the changes made during refactoring.

| Component                                   | Original | After Refactoring      | Reason                                  |
|---------------------------------------------|----------|------------------------|-----------------------------------------|
| <code>Vehicle</code> interface              | Present  | Kept unchanged         | Already well-defined                    |
| <code>Bus</code> , <code>Taxi</code> , etc. | Present  | Minor improvements     | Core logic was correct                  |
| <code>VehicleFactory</code>                 | Absent   | Added (abstract class) | Core of the pattern                     |
| <code>BusFactory</code>                     | Absent   | Added                  | Creates <code>Bus</code> only           |
| <code>TaxiFactory</code>                    | Absent   | Added                  | Creates <code>Taxi</code> only          |
| <code>MotorcycleDeliveryFactory</code>      | Absent   | Added                  | Creates <code>MotorcycleDelivery</code> |
| <code>ElectricScooterFactory</code>         | Absent   | Added                  | Creates <code>ElectricScooter</code>    |
| Main / Client Code                          | Absent   | Added with menu        | Demonstrates pattern usage              |

### 3.2 Change 1 — Vehicle Interface Kept Intact

#### 3.2.1 Original Code

```

1 interface Vehicle {
2     void startTrip();
3     double calculateFare(double distance);
4     void assignRoute(String route);
5     void getVehicleInfo();
6     String getVehicleType();
7 }

```

Listing 2: Original Vehicle interface

#### 3.2.2 Decision

The `Vehicle` interface was **not modified**. It already correctly defines the contract that all vehicle types must follow. This is precisely the **Product Interface** role in the Factory Method Pattern. Keeping it unchanged confirms that the original design had a good foundation — it only lacked the creation layer on top.

### 3.3 Change 2 — Concrete Vehicle Classes Improved

#### 3.3.1 Original Code (example: `Bus`)

```

1 @Override
2 public void getVehicleInfo() {
3     System.out.println("Vehicle: Bus");
4     System.out.println("Capacity: " + capacity);
5     System.out.println("Fare per KM: " + farePerKm);

```

```
6 }
```

Listing 3: Original Bus class — getVehicleInfo()

### 3.3.2 Refactored Code

```
1 @Override
2 public void getVehicleInfo() {
3     System.out.println("Vehicle      : Bus");
4     System.out.println("Capacity      : " + capacity + "
5         passengers");
6     System.out.println("Fare per KM : " + farePerKm + " BDT");
7 }
```

Listing 4: Refactored Bus class — improved output formatting

### 3.3.3 Why This Change?

The output formatting was improved for two reasons. First, aligned labels (using spaces) make the printed information easier to read at a glance, which matters when the system displays multiple vehicle types in sequence. Second, adding units (`passengers`, `BDT`) makes the output self-explanatory — a reader does not need to guess whether capacity is in seats, tonnes, or cubic metres, or whether the fare is in Taka, Dollars, or any other currency.

The core logic (`farePerKm`, `capacity` fields, method implementations) was kept identical, as it was already correct.

## 3.4 Change 3 — Abstract Creator: VehicleFactory (New Addition)

### 3.4.1 Code Added

```
1 abstract class VehicleFactory {
2
3     public abstract Vehicle createVehicle();
4
5     public Vehicle orderVehicle(String route) {
6         Vehicle vehicle = createVehicle();
7         System.out.println("\n[Factory] Creating a "
8             + vehicle.getVehicleType() + "...");
9         vehicle.assignRoute(route);
10        return vehicle;
11    }
12 }
```

Listing 5: Abstract VehicleFactory — the heart of the pattern

### 3.4.2 Why This Change?

This is the most critical addition. `VehicleFactory` is the **Abstract Creator** in the Factory Method Pattern. Here is why each part matters:

- **public abstract Vehicle createVehicle()** — This is the factory method itself. It is declared `abstract` so that each subclass *must* provide its own implementation.

The return type is `Vehicle` (the interface), not a concrete class — this ensures the client code remains decoupled from any specific vehicle type.

- **public `Vehicle orderVehicle(String route)`** — This is a convenience method that uses the factory method. It calls `createVehicle()`, which triggers whichever subclass overrides it, and then automatically assigns a route. This demonstrates the pattern's power: the shared logic lives in the abstract class, while object creation is delegated to subclasses.

Without this class, there is no factory — the system is just a collection of unrelated vehicle classes with no controlled creation strategy.

### 3.5 Change 4 — Concrete Creators: Four Factory Classes (New Additions)

#### 3.5.1 Code Added

```
1 class BusFactory extends VehicleFactory {
2     @Override
3     public Vehicle createVehicle() {
4         return new Bus();
5     }
6 }
7
8 class TaxiFactory extends VehicleFactory {
9     @Override
10    public Vehicle createVehicle() {
11        return new Taxi();
12    }
13 }
14
15 class MotorcycleDeliveryFactory extends VehicleFactory {
16     @Override
17    public Vehicle createVehicle() {
18        return new MotorcycleDelivery();
19    }
20 }
21
22 class ElectricScooterFactory extends VehicleFactory {
23     @Override
24    public Vehicle createVehicle() {
25        return new ElectricScooter();
26    }
27 }
```

Listing 6: Four concrete factory classes

#### 3.5.2 Why This Change?

Each factory class is a **Concrete Creator**. These classes carry all of the `new` keyword usage — they are the *only* place where a specific vehicle type is instantiated.

- **Single Responsibility:** Each factory does exactly one thing. `BusFactory` creates



buses. `TaxiFactory` creates taxis. Nothing more, nothing less.

- **Isolation:** If the `Bus` class constructor changes (e.g., requires a new parameter), only `BusFactory` needs updating — no other part of the system is affected.
- **Extensibility:** To add a `Train` vehicle, a developer simply adds a `Train` class and a `TrainFactory` class. Zero existing code is modified.

## 3.6 Change 5 — Client Code with Menu (New Addition)

### 3.6.1 Code Added

```
1 public static VehicleFactory getFactory(int choice) {
2     switch (choice) {
3         case 1: return new BusFactory();
4         case 2: return new TaxiFactory();
5         case 3: return new MotorcycleDeliveryFactory();
6         case 4: return new ElectricScooterFactory();
7         default: return null;
8     }
9 }
```

Listing 7: Client code — `getFactory()` helper method

```
1 VehicleFactory factory = getFactory(vehicleChoice);
2 Vehicle myVehicle = factory.orderVehicle(route);
3 myVehicle.getVehicleInfo();
4 double fare = myVehicle.calculateFare(distance);
5 myVehicle.startTrip();
```

Listing 8: Client code — using the factory (no direct `new Vehicle()`)

### 3.6.2 Why This Change?

The original code had no `main()` method or client code at all. The refactored version introduces a complete interactive main class that demonstrates the pattern correctly.

The `getFactory()` method acts as a **factory selector**. It is the only method that maps a user's input number to a factory class. Critically:

- The `main()` method **never calls `new Bus()` or `new Taxi()`**. All construction is delegated to factories.
- The variable `myVehicle` is typed as `Vehicle` (the interface), not as any concrete class. This means the rest of the loop body works identically regardless of which vehicle was selected.
- Adding a new vehicle (e.g., `Train`) only requires: (1) a `Train` class, (2) a `TrainFactory` class, and (3) one new `case` line in `getFactory()`. The while loop, the fare calculation, the trip start — all remain untouched.

## 4 Comparison: Before vs. After Refactoring

| Before Refactoring                                                    | After Refactoring                                      |
|-----------------------------------------------------------------------|--------------------------------------------------------|
| No main class or client code                                          | Full interactive main class with menu                  |
| No object creation strategy                                           | Factory Method Pattern applied                         |
| <code>new Bus()</code> , <code>new Taxi()</code> scattered everywhere | All <code>new</code> calls isolated in factory classes |
| Adding a new vehicle requires editing main logic                      | Adding a new vehicle only requires a new class         |
| No separation of concerns                                             | Creation logic separated from business logic           |
| Tightly coupled to concrete classes                                   | Client depends only on abstract types                  |
| Difficult to test in isolation                                        | Each factory and vehicle is independently testable     |
| Violates Open/Closed Principle                                        | Follows Open/Closed Principle                          |

## 5 Design Principles Applied

The refactoring enforces three fundamental object-oriented design principles:

### 1. Open/Closed Principle (OCP)

Classes are open for extension (new factories can be added) but closed for modification (existing factories and the abstract class do not need to change).

### 2. Single Responsibility Principle (SRP)

Each class has one reason to change. `Bus` changes only if bus behaviour changes. `BusFactory` changes only if bus creation changes. `VehicleFactory` changes only if the shared ordering logic changes.

### 3. Dependency Inversion Principle (DIP)

High-level modules (`main`) depend on abstractions (`VehicleFactory`, `Vehicle`), not on concrete implementations (`Bus`, `Taxi`).

## 6 Conclusion

The Smart Transportation Management System was successfully refactored from an unstructured prototype into a clean, scalable system using the **Factory Method Design Pattern**. The original vehicle classes provided a solid foundation through the `Vehicle` interface, but the system lacked any controlled creation mechanism.

By introducing the abstract `VehicleFactory` class and four corresponding concrete factory classes, the system now separates object creation from business logic entirely. The client code (the `main` method) operates only on abstract types, making the system easy to read, extend, and maintain.

The most tangible practical benefit is extensibility: if the transportation company decides to add a new vehicle type — say, a `Ferry` or a `Helicopter` — a developer needs to write exactly two new classes (`Ferry` and `FerryFactory`) and add one case to the factory

selector. Not a single line of existing, tested code needs to be changed.

This is the promise of the Factory Method Design Pattern: *grow your system by adding, not by modifying.*